Krickora - Self-Service Keypad Entry

DIY Build Plan & Specification | Rev 0.1 (draft) | 2026-06-05 | Jaycar (AU) sourcing

Read first - assumptions. No written hardware spec existed in the repo, so this plan is reconstructed from the booking / access-code software (access-code.ts, Convex + Google Calendar + Stripe) and our design discussion. Items still to be decided are marked [DECISION]. Confirm flagged items before ordering or installing. Prices are indicative AUD only - verify current Jaycar pricing/stock.

1. Intent & scope

Customers book and pay online, the system issues a numeric door code bound to that booking, the customer enters the code at a keypad at the facility entry, the door releases and their booked bay (e.g. bay 4) powers up for the booking window. This plan covers the physical keypad entry controller, its wiring, firmware, the backend validation endpoint it relies on, and the reliability/safety engineering to keep it running long-term.

2. Architecture - dumb edge, smart core

The design goal is maximum uptime. Keep the device at the door as simple and stateless as possible, and put the decision logic on the well-powered, UPS-backed server.

- **Edge (at the door): ESP32 + ESPHome.** Scans the keypad, drives the door relay, shows status. No OS, no SD card, boots in under 1s, recovers instantly from power loss. Native Home Assistant integration.
- **Core (server): Home Assistant.** Holds booking state, validates the code against the active booking window, decides open/deny, logs every attempt, triggers the bay (lights/power/door).
- **Source of truth: Convex backend.** HA validates against a hardened /door/validate endpoint and keeps a local offline cache of today's valid codes so entry still works if the internet drops.

Why not a Raspberry Pi at the door? For a single-purpose door controller a Pi is usually LESS reliable, not more: a full Linux OS on an SD card is the #1 field-failure mode (corruption on power loss) - exactly the failure that locks customers out. A Pi is right only if the door device itself must do heavy local compute (touchscreen, camera/ANPR, large offline DB). If a Pi is required, harden it: boot from SSD/eMMC (not SD), read-only root filesystem (overlayfs), hardware watchdog, and UPS with clean shutdown.

3. Bill of materials (Jaycar AU)

Confirmed cat. numbers from Jaycar. Items marked "confirm" are common Jaycar lines - verify the exact cat. no. online/in-store.

#	Item	Jaycar Cat.	Qty	AUD	Notes
1	Duinotech ESP32 Main Board (WiFi/BT)	XC3800	1 (+1 spare)	~$30	Edge controller. Keep a pre-flashed spare for instant swap.
2	12-Key Numeric Keypad (3x4 matrix)	SP0770	1	~$8	0-9, *, #. 7-wire matrix.
3	4-Channel 5V Relay Module	XC4441	1	~$15	Over-spec: 1 ch strike, spare ch for roller-door/bay. Opto-isolated, 10A.
4	Electric Door Strike 12VDC, Fail-Secure (narrow)	LA5077	1	~$40	[DECISION] Locked on power loss. See sec.10 egress. Fail-safe alt: LA5079/LA5081.
5	Non-Contact IR Exit Switch (Request-to-Exit)	LA5187	1	~$30	Touchless egress. Or reuse existing green exit button.
6	12V regulated PSU (>=2A)	confirm	1	~$30	Powers strike + ESP32 (via buck). Size for strike inrush.
7	DC-DC Buck Converter (12V to 5V)	XC4514	1	~$10	Feeds ESP32 5V from 12V rail. Or a quality USB supply.
8	Flyback diode 1N4004 (strike coil)	ZR1004	1	~$1	Across strike coil, protects relay/electronics. Essential.
9	IP-rated project enclosure	confirm (HB-series)	1	~$20	Houses ESP32 + relay + buck. Keypad on door face.
10	Wire, ferrules, screw terminals, standoffs	assorted	-	~$20	Use screw terminals (not breadboard) for a permanent install.

Optional over-spec add-ons: small UPS / 12V battery + charger for the door controller so entry survives a mains blip; buzzer + RGB status LED; door-position reed switch for held-open / forced-door detection.

4. Wiring - keypad & relay to ESP32

The SP0770 3x4 keypad uses 7 lines (4 rows + 3 columns). Suggested ESP32 GPIO map (avoids strapping/boot pins):

Signal	GPIO	Signal	GPIO
Row 1	GPIO 13	Col 1	GPIO 26
Row 2	GPIO 12	Col 2	GPIO 25
Row 3	GPIO 14	Col 3	GPIO 33
Row 4	GPIO 27	Relay IN1 (strike)	GPIO 32
Status LED	GPIO 4	Buzzer	GPIO 2
Exit switch (REX) in	GPIO 35 (input-only)	Spare relay IN2 (roller)	GPIO 19

Power/strike path: 12V PSU -> relay common/NO contacts -> door strike -> back to PSU, with the 1N4004 flyback diode across the strike coil (cathode/band to +12V). 12V PSU -> XC4514 buck -> 5V -> ESP32. Common ground between ESP32, relay board and buck.

Mains / 12V isolation: the relay CONTACTS switch the 12V strike circuit; the relay COIL side is opto-isolated from the ESP32. Keep the 12V door circuit physically separated from logic wiring. Do not switch mains with this board.

5. ESPHome firmware (edge)

Representative ESPHome config - scans the keypad, collects a code, hands it to Home Assistant, and exposes the strike as something HA can pulse:

```
esphome:
name: door-keypad-entry
esp32:
board: esp32dev
wifi:
ssid: !secret wifi_ssid
password: !secret wifi_password # static IP recommended
api: # native encrypted HA link
encryption:
key: !secret api_key
logger:
ota:

matrix_keypad:
id: kpad
rows: { pins: [13, 12, 14, 27] }
columns: { pins: [26, 25, 33] }
keys: "123456789*0#"

key_collector:
- id: code_entry
source_id: kpad
min_length: 4
max_length: 6
end_key: "#"
clear_key: "*"
on_result:
- homeassistant.event:
event: esphome.door_code_entered
data: { code: !lambda 'return x;' }

switch:
- platform: gpio
pin: 32
id: door_strike
name: "Door Strike"

button:
- platform: template
name: "Release Door"
on_press:
- switch.turn_on: door_strike
- delay: 4s
- switch.turn_off: door_strike
```

Validation happens on HA/Convex, not on the ESP32 - the door device never holds booking logic, so it cannot be reverse-engineered for codes and stays trivially simple.

6. Home Assistant logic (core)

1. HA listens for esphome.door_code_entered.
2. HA calls Convex /door/validate with the code (and device auth). On network failure, HA falls back to its local cache of today's valid codes.
3. If valid and within the booking window, HA presses Release Door on the ESP32 and powers up the booked bay (reusing the calendar-driven bay automations).
4. Every attempt (allow/deny) is logged with timestamp and code masked.

7. Backend - /door/validate endpoint (Convex)

This endpoint does not exist yet (http.ts has only health/auth/Stripe). It must be added:

```
POST /door/validate
Headers: X-Door-Token: <shared secret per device>
Body: { "code": "4821" }

-> 200 { "allow": true, "bay": "bay_4", "bookingId": "..." }
-> 200 { "allow": false, "reason": "no_active_booking" }
-> 429 { "allow": false, "reason": "rate_limited" }

Server checks:
- booking exists with this code
- now within [start - preGrace, end + postGrace]
- booking is paid and not cancelled
- device token valid
- rate-limit / lockout counters OK
```

8. Backend defects to fix first (security & reliability)

WARNING - a keypad is only as trustworthy as what it validates against. access-code.ts currently has issues that make it unsafe to drive a physical door as-is:

- **Codes live in an in-memory Set (activeCodes).** Resets on every redeploy and is not shared across serverless instances - invalidateAccessCode() and the 24h setTimeout cleanup will not reliably work. Fix: store the code on the booking row in the Convex DB, indexed, validated server-side against the booking time window.
- **4-digit codes = 9,000 combinations** - brute-forceable for physical entry. Fix: enforce rate-limiting + lockout (e.g. 5 tries then 60s cooldown), keep codes valid only during the booking window + grace, and consider 6-digit for new bookings.
- **No time-binding at validation.** A code must never open the door outside its booking window - enforce server-side.
- **Log & alert** on repeated failures (possible tampering).

These are prerequisites, not polish - schedule them before go-live.

9. Offline / fail-safe behaviour [DECISION]

- **Internet down:** HA validates against its local cache of today's codes; entry still works. Cache refreshes from Convex every few minutes.
- **HA down:** decide whether the door fails locked (secure) or has an attendant override. A small 12V battery keeps ESP32 + strike alive through brief outages.
- **Power-on behaviour:** on restore, ESP32 boots in under 1s; relay defaults de-energised (door locked for a fail-secure strike).

10. Egress & fire-safety compliance - must confirm

LIFE SAFETY. A powered entry on an occupied facility carries building-code obligations (NCC/BCA, AS 1428, fire egress). People inside must ALWAYS be able to exit regardless of power or network state.

- **Fail-secure vs fail-safe** is a safety decision, not just security - confirm against local code. Fail-safe strikes (LA5079/LA5081) unlock on power loss; fail-secure (LA5077) stays locked.
- Provide free mechanical egress (handle/push-bar) and/or a fail-safe exit path independent of the electronics. The IR exit switch / green exit button supports request-to-exit but must not be the only way out.
- Have a licensed installer / building surveyor sign off before commissioning.

11. Power & UPS

Tie the door controller into the facility UPS plan. The 12V door rail can run from the UPS-backed supply (or its own small 12V battery + charger) so a mains blip never locks the door. The HA server should be on the UPS with

BIOS "restore on AC power loss = On" so the core recovers automatically after an outage.

12. Installation steps

1. Bench-build: wire keypad + relay + buck to ESP32 in the enclosure; flash ESPHome; confirm it appears in HA.
2. Bench-test the full chain (code -> HA -> validate -> relay pulse) before touching the door.
3. Fit the strike to the door jamb; wire the 12V strike circuit with flyback diode; verify fail-secure/fail-safe behaviour matches the egress decision.
4. Mount keypad on the door face, controller enclosure inside, exit switch on the egress side.
5. Set ESP32 to a static IP; lock down the device token; enable HA logging.
6. Deploy backend /door/validate + DB code storage + rate-limiting (sec.7-8).
7. Installer/surveyor egress sign-off, then go live.

13. Commissioning test checklist

Test	Expected
Valid code, within booking window	Door releases ~4s; bay powers up; entry logged
Valid code, outside booking window	Denied; logged
Wrong code x5	Lockout/cooldown triggers; alert raised
Internet pulled	Valid code still works via local cache
Mains pulled to controller	Strike behaves per egress decision; ESP32 recovers on restore
Exit switch / mechanical egress	Always opens from inside, powered or not
Cancelled booking code	Denied immediately after cancellation propagates

14. Open decisions before ordering

1. [DECISION] Door hardware: electric strike on a personnel door (this plan) vs triggering the existing roller-door operator - or both (spare relay channel reserved).
2. [DECISION] Single shared entry vs a keypad per bay.
3. [DECISION] Fail-secure vs fail-safe (driven by sec.10 egress/compliance).
4. [DECISION] Full offline operation vs brief ride-through only.
5. [DECISION] Reuse existing green exit button vs new IR exit switch (LA5187).